

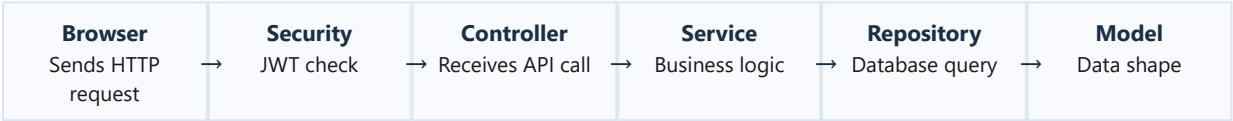


CafeBook — Java Logic Explained

All 19 Java files, what each one does, and the logic inside them

CafeBook Backend | Spring Boot 3.2 | Java 21 | May 2026

Every request from the browser travels through 4 layers before a response comes back:
Controller (receives the request) → **Service** (applies business rules) → **Repository** (reads/writes database) → **Model** (defines the data shape)



LAYER 1 **App Startup — 2 files**

1 **BookingApplication.java** Entry Point

- Starts the entire application. One method: `main()`
- Calls `SpringApplication.run()` which boots up the web server on port 8080
- `@SpringBootApplication` tells Spring to scan all files in the project and auto-configure everything
- Think of it as the *power-on switch* — nothing runs without this

2 **DataInitializer.java** Seed Data

- Runs **once after the server starts** (implements `CommandLineRunner`)
- Check before insert** — only creates data if none exists yet. Safe to restart.
- Creates the admin account: `admin@cafe.com` / `admin123`
- Creates 6 seating zones (Window Side, Cosy Corner, Patio, etc.)
- Creates 12 tables (W1, W2, C1, C2 ... L2) and links each to its zone

LAYER 2 **Models — 4 files** (Define what data looks like in the database)

3 **User.java** Model / Entity

- Represents a row in the `users` database table
- Fields: id, name, email (unique), phone, password, role, dietaryNotes, accessibilityNeeded, createdAt
- Role logic:** enum `ADMIN` or `GUEST` — controls what the user can access
- `@JsonIgnore` on `getPassword()` — password is **never sent** to the browser in any API response
- Default role is `GUEST` for all new registrations

4 **Zone.java** Model / Entity

- Represents a seating area inside the cafe (e.g. "Window Side")
- Fields: id, name, description
- Tables are linked to zones — one zone can have many tables
- Simple lookup data; never changed by users — only set up once by `DataInitializer`

5 **RestaurantTable.java** Model / Entity

6 **Booking.java** Model / Entity

- Represents one physical seat/table in the cafe
- Fields: id, tableNumber (e.g. W1), minCapacity, maxCapacity, isAvailable, isAccessible, type, specialFeatures, zone
- **Type logic:** enum — STANDARD , GROUP , PRIVATE , or MERGED
- isAvailable flag — admin can toggle this to block a table from being booked
- isAccessible flag — marks wheelchair-friendly tables
- Linked to a Zone via @ManyToOne (many tables belong to one zone)

- Represents one reservation made by a customer
- Fields: id, user, table, bookingDate, bookingTime, partySize, status, specialOccasion, specialNotes, depositAmount, createdAt
- **Status logic:** enum — PENDING → CONFIRMED → COMPLETED or CANCELLED / NO_SHOW
- Links to both User and RestaurantTable via @ManyToOne
- Default status is PENDING — admin must change it to CONFIRMED

LAYER 2b**DTO (Data Transfer Object) — 1 file** (Shape of incoming request data)**7 BookingRequest.java**

DTO

- A simple container that holds the data sent by the browser when creating a booking
- Fields: tableId, bookingDate, bookingTime, partySize, specialOccasion, specialNotes, depositAmount
- **Why separate from Booking.java?** — The browser sends only what it knows (no user ID needed — server gets that from the JWT token). Keeping input data separate from the database model is cleaner and safer.
- No logic here — just fields with getters and setters

LAYER 3

Repositories — 4 files (Talk to the database — read and write)

Repositories extend `JpaRepository` — Spring automatically writes all standard SQL queries (`findAll`, `findById`, `save`, `delete`) so you don't have to. You only write custom queries for special needs.

8 UserRepository.java

Repository

- Provides all standard DB operations for the `users` table (auto-generated)
- **Custom logic:** `findByEmail(email)` — looks up a user by their email address
- Used by: `AuthService` (login/register), `JwtFilter` (token verification), `BookingService`
- Returns `Optional<User>` — safely handles the case when the email does not exist

9 ZoneRepository.java

Repository

- Provides all standard DB operations for the `zones` table
- No custom methods needed — only used by `DataInitializer` to check `count()` (are zones already seeded?)
- Zones are read-only after first startup — never updated by users or admin

10 BookingRepository.java

Repository

- Provides all standard DB operations for the `bookings` table
- **Custom logic:** `findByUserOrderByCreatedAtDesc(user)`
- Fetches all bookings for one specific user, sorted with newest first
- Spring reads the method name and auto-generates the SQL — no query needed

11 TableRepository.java

Repository + Custom Query

- Contains the **most important query** in the entire project
- **Logic:** `findAvailable(date, time, partySize, excluded)`
- Step 1 — Filter: only tables where `isAvailable = true`
- Step 2 — Filter: only tables where `maxCapacity ≥ partySize` (table fits the group)
- Step 3 — Exclude: any table that already has a booking on that exact date + time, where the booking is NOT cancelled or no-show
- Result: only truly free, fitting tables are returned
- Uses `Collection<BookingStatus>` typed enum — required for Hibernate 6 compatibility

LAYER 4

Services — 2 files (Business rules — where decisions are made)**12 AuthService.java**

Service — Login & Register

- **register() logic:** (1) Check if email already exists — if yes, throw error "Email already registered" (2) Hash the password using BCrypt — never store plain text (3) Save the new user to DB with role = GUEST (4) Create a JWT token with the user's email inside (5) Return both the token and user object to the browser
- **login() logic:** (1) Find user by email — if not found, throw "Invalid email or password" (2) Compare the typed password against the stored BCrypt hash — if wrong, same error (3) Create a JWT token and return it with the user object

- The same generic error message is used whether the email is wrong or the password is wrong — this prevents attackers from discovering which emails are registered

13 BookingService.java

Service — Booking Rules

- **create() logic:** Find user by email from JWT → find table by tableId → build a new Booking object with all fields → set status = PENDING → save to DB → return saved booking
- **getMyBookings() logic:** Find user by email → call `findByUserOrderByCreatedAtDesc()` → return list sorted newest first
- **cancel() logic (security check!):** Load booking → verify `booking.user.email == logged-in email` → if different user, throw "Unauthorized" → otherwise set status = CANCELLED and save. Prevents users from cancelling someone else's booking.
- **getAllBookings():** Returns every booking in the DB — admin use only (access controlled in SecurityConfig)
- **updateStatus():** Load booking by ID → set new status → save. Used by admin to change PENDING → CONFIRMED, etc.

LAYER 5 Security & Config — 2 files (Guards the application)

14 JwtFilter.java

Security — Token Handler

- Runs on **every single HTTP request** before it reaches any controller
- **createToken() logic:** Builds a signed JWT with the user's email as the subject. Sets expiry to 24 hours. Signs it using a secret key from `application.properties`. Called once after login/register.
- **doFilterInternal() logic — 5 steps:**
 - (1) Read the `Authorization` header from the request
 - (2) If header starts with "Bearer ", extract the token string
 - (3) Decode the token → extract the email stored inside
 - (4) Look up that email in the database → get user's role
 - (5) Set the user as "authenticated" in Spring's security context for this request
- If the token is invalid, expired, or missing — the request continues as **unauthenticated** (Spring Security then blocks it if the route requires login)

15 SecurityConfig.java

Security — Access Rules

- Defines the access rules (who can reach which URL)
- **Public (no login needed):** HTML pages, CSS, JS files, H2 console, `/api/auth/**` (login/register), `/api/tables` (browse tables)
- **Admin only:** Any URL under `/api/admin/**` — blocked unless user has role ADMIN
- **Everything else:** requires a valid JWT token (must be logged in)
- Disables CSRF protection — not needed for stateless JWT APIs
- Disables frame headers — needed so the H2 database console works in the browser
- Sets session policy to STATELESS — no server-side session, only JWT tokens
- Registers `BCryptPasswordEncoder` so services can use it for hashing

LAYER 6

Controllers — 4 files (Receive HTTP requests from the browser — the front door)

Controllers do **not** contain business logic — they only receive the request, call a service or repository, and send back a response. They are the bridge between the browser and the backend.

16 AuthController.java

Controller — /api/auth

- Handles two routes — both are public (no login needed)
- **POST /api/auth/register:** Receives a JSON body with name, email, phone, password, dietaryNotes, accessibilityNeeded → passes to `AuthService.register()` → returns token + user, or 400 Bad Request with error message
- **POST /api/auth/login:** Receives JSON body with email and password → passes to `AuthService.login()` → returns token + user, or 400 Bad Request with error message
- Uses `Map<String, Object>` for the request body — no separate DTO class needed
- Error handling: catches `RuntimeException` and returns it as a JSON error message with HTTP 400

17 BookingController.java

Controller — /api/bookings

- All 3 routes require a valid JWT token (login required)
- Uses `Authentication auth` parameter — Spring automatically injects the logged-in user's info (set by `JwtFilter`). `auth.getName()` returns the user's email.
- **POST /api/bookings:** Creates a new booking. Reads the email from the JWT (not from the request body — user cannot fake their identity). Passes `BookingRequest` + email to `BookingService.create()`
- **GET /api/bookings/my:** Returns all bookings belonging to the logged-in user only
- **PATCH /api/bookings/{id}/cancel:** Cancels a specific booking. Passes the booking ID + user email to `BookingService.cancel()`, which verifies ownership before cancelling

18 TableController.java

Controller — /api/tables

- Both routes are **public** — no login needed (customers need to browse tables before registering)
- **GET /api/tables:** Returns all tables in the database. Used by admin dashboard to list every table.
- **GET /api/tables/available?date=&time=&partySize=:** Calls `TableRepository.findAvailable()` with the search parameters. Returns only tables that are free, available, and fit the group size on that exact date and time.
- Calls the repository directly — no service layer needed for these simple lookups

19 AdminController.java

Controller — /api/admin (ADMIN only)

- All routes are **admin-only** — blocked by `SecurityConfig` for regular users
- **GET /api/admin/bookings:** Returns every booking from every customer. Calls `BookingService.getAllBookings()`
- **PATCH /api/admin/bookings/{id}/status?status=:** Changes a booking's status (e.g. PENDING → CONFIRMED). Passes ID + status to `BookingService.updateStatus()`

- **POST /api/admin/tables:** Adds a new table to the database. Receives the full table object in the request body and saves it directly via `TableRepository`
- **PATCH /api/admin/tables/{id}/availability?available=:** Loads a table by ID → sets its `isAvailable` flag to true or false → saves. Used to block/unblock a table for bookings.
- **DELETE /api/admin/tables/{id}:** Permanently deletes a table. Returns HTTP 204 No Content on success.

SUMMARY

All 19 Files at a Glance

#	File	Layer	Key Logic in One Line
1	BookingApplicationStartup		Starts the Spring Boot server
2	DataInitializer	Startup	Creates admin, zones, and 12 tables on first run
3	User	Model	User shape: email unique, password hidden, role ADMIN/GUEST
4	Zone	Model	Seating area with name and description
5	RestaurantTable	Model	Table shape: number, capacity range, availability, zone link
6	Booking	Model	Reservation shape: links user+table, status starts PENDING
7	BookingRequest	DTO	Input data container for creating a booking (no user ID — server adds that)
8	UserRepository	Repository	DB access for users; custom: findByEmail()
9	ZoneRepository	Repository	DB access for zones; only used at startup
10	BookingRepository	Repository	DB access for bookings; custom: find by user, newest first
11	TableRepository	Repository	Custom JPQL query: finds free tables by date, time, party size
12	AuthService	Service	Register: check duplicate email, hash password, create token. Login: verify password, create token.
13	BookingService	Service	Create booking; get my bookings; cancel (with ownership check); admin update status
14	JwtFilter	Security	Runs on every request: reads JWT token, decodes email, sets user as authenticated
15	SecurityConfig	Security	Public vs login-required vs admin-only URL rules; registers BCrypt encoder
16	AuthController	Controller	POST /api/auth/register and /login — public endpoints
17	BookingController	Controller	POST create, GET my bookings, PATCH cancel — all need login
18	TableController	Controller	GET all tables, GET available tables — public
19	AdminController	Controller	All bookings, update status, add/toggle/delete tables — admin only